



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1 ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Обработка больших чисел

Вариант № 5

Студент: Крылов Владислав Сергеевич

Группа: ИУ7-32Б

Преподаватель:

Барышникова М. Ю.

2024 г.

1. Описание условия задания

Смоделировать операцию умножения двух действительных чисел в форме:

$\pm m.nE\pm K$,

где:

- длина мантиссы первого числа ($m+n$) — до 35 значащих цифр,
- длина мантиссы второго числа — до 40 значащих цифр,
- величина порядка K — до 5 значащих цифр.

Результат выдать в нормализованной форме:

$\pm 0.m1E\pm K1$,

где:

- $m1$ — до 40 значащих цифр,
- $K1$ — до 5 значащих цифр.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные: два числа в строковом формате длиной до 100 символов.

Формат чисел:

- Знак мантиссы (опционально, по умолчанию '+'),
- Целая и дробная части (разделены точкой),
- Экспонента (символ 'E', знак экспоненты, целое число до 5 цифр).

Ограничения:

- Мантисса первого числа ≤ 35 цифр,
- Мантисса второго числа ≤ 40 цифр,
- Экспонента ≤ 5 цифр.

Результат: число в нормализованной форме $\pm 0.m1E\pm K1$, где:

где:

- $m1$ — до 40 значащих цифр,
- $K1$ — до 5 значащих цифр.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: `./app.exe`

Ошибки:

- Пустой ввод;
- Неверный формат;
- Превышение длины мантиссы;
- Превышение длины экспоненты;
- Переполнение/исчезновение порядка.

Описание задачи, реализуемой программой:

1. Принять из консоли первое число.
2. Проверить корректность ввода и считать его в структуру числа.
3. Принять из консоли второе число.
4. Проверить корректность ввода и считать его в структуру числа.
5. Умножить числа и проверить корректность результата.
6. Вывести результат.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мною была реализована следующая структура данных:

c
<pre>typedef struct { char sign_mantissa; char mantissa[MAX_INPUT_LEN + 1]; char sign_exponent; int exponent; } number;</pre>

Данная структура данных описывает число в виде знака мантиссы, мантиссы, знака экспоненты и экспоненты.

- **sign_mantissa** – символ знака числа (либо “+”, либо “-”);
- **mantissa** – строка, состоящая из значащих цифр числа;
(MAX_INPUT_LEN = 100)
- **sign_exponent** – символ знака экспоненты (либо “+”, либо “-”);
- **exponent** – целочисленная экспонента;

4. Алгоритмы

Псевдокод функции **parse_number**

Псевдокод функции parse_number

Вход:

number_str - строка для парсинга

number - структура для хранения числа

max_size_mantissa - максимальный размер мантиссы

Выход:

number - заполненная структура числа

Код возврата

len = длина number_str

Если len = 0:

 Вернуть ERROR_EMPTY

status = 0 # 0-знак, 1-мантисса, 2-знак экспоненты, 3-экспонента

nools = 0

has_digit = 0

len_mantissa = 0

pos_dot = -1

number.exponent = 0

number.sign_mantissa = '+'

number.sign_exponent = '+'

Для каждого символа c в number_str:

 Если status = 0: # Обработка знака

 Если c = '+':

 status = 1

 number.sign_mantissa = '+'

 Иначе если c = '-':

 status = 1

 number.sign_mantissa = '-'

 Иначе если c = '0':

 status = 1

 has_digit = 1

 nools = 1

 number.mantissa[len_mantissa] = c

 len_mantissa = len_mantissa + 1

 Иначе если c - цифра:

 status = 1

 has_digit = 1

 number.mantissa[len_mantissa] = c

 len_mantissa = len_mantissa + 1

 Иначе если c = '.':

 status = 1

 nools = 0

```
pos_dot = 0
Иначе:
    Вернуть ERROR_INPUT
```

Иначе если status = 1: # Обработка мантиссы

```
Если c = 'E':
    status = 2
Иначе если c - цифра:
    Если nools = 1 И c ≠ '0':
        nools = 0
        number.mantissa[len_mantissa] = c
        len_mantissa = len_mantissa + 1
        Если len_mantissa > max_size_mantissa:
            Вернуть ERROR_LONG_MANTISSA
```

```
Иначе если nools = 0:
    number.mantissa[len_mantissa] = c
    len_mantissa = len_mantissa + 1
    Если len_mantissa > max_size_mantissa:
        Вернуть ERROR_LONG_MANTISSA
```

```
Иначе если nools = 1 И c = '0' И has_digit = 0:
    has_digit = 1
    number.mantissa[len_mantissa] = c
    len_mantissa = len_mantissa + 1
```

```
Иначе если c = '.':
    Если pos_dot = -1:
        nools = 0
        pos_dot = len_mantissa
    Иначе:
        Вернуть ERROR_INPUT
```

```
Иначе:
    Вернуть ERROR_INPUT
```

Иначе если status = 2: # Обработка знака экспоненты

```
Если c = '+':
    status = 3
    number.sign_exponent = '+'
```

```
Иначе если c = '-':
    status = 3
    number.sign_exponent = '-'
```

```
Иначе если c - цифра:
    status = 3
    number.exponent = c - '0'
```

```
Иначе:
    Вернуть ERROR_INPUT
```

Иначе если status = 3: # Обработка экспоненты

```
Если c — цифра:
    number.exponent = number.exponent * 10 + (c - '0')
    Если number.exponent > 99999:
        Вернуть ERROR_LONG_EXPONENTA
```

```
Иначе:
    Вернуть ERROR_INPUT
```

```

Если len_mantissa = 0:
    Вернуть ERROR_INPUT
Если len_mantissa > max_size_mantissa:
    Вернуть ERROR_LONG_MANTISSA

number.mantissa[len_mantissa] = '\0'

Если pos_dot = -1:
    pos_dot = len_mantissa

Если number.sign_exponent = '-':
    number.exponent = -number.exponent

number.exponent = number.exponent + pos_dot

Если number.exponent ≥ 0:
    number.exponent = number.exponent
    number.sign_exponent = '+'
Иначе:
    number.exponent = -number.exponent
    number.sign_exponent = '-'

Вызвать delete_lead_zeros(number)

Если number.exponent < 0:
    number.exponent = -number.exponent
    Если number.sign_exponent = '+':
        number.sign_exponent = '-'
    Иначе:
        number.sign_exponent = '+'

Вызвать delete_end_zeros(number)

Вернуть ОК

```

Псевдокод функции **multiplication**

Псевдокод функции multiplication

Вход:

a - первое число

b - второе число

Выход:

result - результат умножения

Код возврата

result.sign_mantissa = '+' если a.sign_mantissa = b.sign_mantissa, иначе '-'

result.sign_exponent = '+'

result.exponent = 0

```
exp_a = a.exponent если a.sign_exponent = '+', иначе -a.exponent
exp_b = b.exponent если b.sign_exponent = '+', иначе -b.exponent
result.exponent = exp_a + exp_b - 1
```

```
Если result.exponent  $\geq$  0:
    result.exponent = result.exponent
    result.sign_exponent = '+'
```

```
Иначе:
    result.exponent = -result.exponent
    result.sign_exponent = '-'
```

```
len_a = длина a.mantissa
len_b = длина b.mantissa
```

```
Если (len_a = 1 И a.mantissa[0] = '0') ИЛИ (len_b = 1 И b.mantissa[0] = '0'):
    result.mantissa = "0"
    result.exponent = 0
    result.sign_exponent = '+'
    result.sign_mantissa = '+'
    Вернуть ОК
```

```
Создать temp_result[MAX_LEN_MANTISSA * 2] = {0}
```

```
Для i от len_a - 1 до 0:
```

```
    Для j от len_b - 1 до 0:
```

```
        product = (a.mantissa[i] - '0') * (b.mantissa[j] - '0')
```

```
        pos = (len_a - 1 - i) + (len_b - 1 - j)
```

```
        temp_result[pos] = temp_result[pos] + product
```

```
    k = pos
```

```
    Пока temp_result[k]  $\geq$  10:
```

```
        temp_result[k + 1] = temp_result[k + 1] + temp_result[k] / 10
```

```
        temp_result[k] = temp_result[k] % 10
```

```
        k = k + 1
```

```
result_len = MAX_LEN_MANTISSA * 2
```

```
Пока result_len > 0 И temp_result[result_len - 1] = 0:
```

```
    result_len = result_len - 1
```

```
Если result_len = 0:
```

```
    result.mantissa = "0"
```

```
    Вернуть ОК
```

```
Если result_len > len_a + len_b - 1:
```

```
    result.exponent = result.exponent + 1
```

```
Создать temp_str[result_len + 1]
```

```
Для i от 0 до result_len - 1:
```

```
    temp_str[i] = temp_result[result_len - 1 - i] + '0'
```

```
temp_str[result_len] = '\0'
```

```
Если result_len > MAX_LEN_RESULT:
```

Если temp_str[MAX_LEN_RESULT] ≥ '5':

carry = 1

Для i от MAX_LEN_RESULT - 1 до 0 И carry ≠ 0:

digit = (temp_str[i] - '0') + carry

temp_str[i] = (digit % 10) + '0'

carry = digit / 10

Если carry ≠ 0:

Для i от MAX_LEN_RESULT до 1:

temp_str[i] = temp_str[i - 1]

temp_str[0] = '1'

result.exponent = result.exponent + 1

Если result.exponent > 99999:

Вернуть ERROR_OVERFLOW

temp_str[MAX_LEN_RESULT] = '\0'

Скопировать temp_str в result.mantissa

result.mantissa[MAX_LEN_RESULT] = '\0'

Вызвать delete_lead_zeros(result)

Вызвать delete_end_zeros(result)

Если result.exponent > 99999:

Вернуть ERROR_OVERFLOW

Если result.exponent < -99999:

Вернуть ERROR_UNDERFLOW

Вернуть ОК

5. Тесты

Позитивные тесты:

1. $1 * 1$;
2. $0 * 0$;
3. Переполнение разряда;
4. Положительное целое число на отрицательное целое число;
5. Умножение целых отрицательных чисел;
6. Умножение вещественных чисел в экспоненциальной форме;
7. Умножение вещественных чисел не в экспоненциальной форме;
8. Умножение чисел с лидирующими незначащими нулями;
9. Умножение чисел с конечными незначащими нулями;
10. Умножение чисел с различными знаками экспоненты;
11. Умножение числа с 35 символами мантииссы на число с 40 символами мантииссы (округление);
12. Умножение чисел с пятизначной экспонентой;

Негативные тесты:

1. Пустой ввод;
2. Невалидный ввод;
3. Несколько знаков мантииссы;
4. Несколько точек;
5. Несколько знаков экспоненты;
6. Мантиисса > 35 символов при вводе первого числа;
7. Мантиисса > 40 символов при вводе второго числа;
8. Экспонента > 5 символов при вводе числа;
9. Отсутствие мантииссы перед экспонентой;
10. Несколько знаков (+-) перед экспонентой;
11. Результирующий порядок > 99999 ;
12. Результирующий порядок < -99999 ;

6. Выводы

Итак, можно сделать вывод, что для того чтобы обрабатывать числа превосходящие разрядную сетку, можно применять алгоритмы для работы с числом поразрядно. Однако для этого программист должен использовать другие способы хранения чисел и разработать свои функции для работы с числами в данном виде.

7. Ответы на контрольные вопросы

1. Каков возможный диапазон чисел, представляемых в ПК?

Для целых чисел: Диапазон ограничен количеством разрядов (битов), выделяемых под хранение числа.

- 16 разрядов: до $2^{16} - 1 = 65\,535$
- 32 разряда: до $2^{32} - 1 = 4\,294\,967\,295$
- 64 разряда: до $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$

Для вещественных чисел с плавающей точкой: Диапазон ограничен длиной порядка. При максимальном выделении 11 разрядов под порядок возможные значения чисел находятся в диапазоне от $3.6E-4951$ до $1.1E+4932$.

2. Какова возможная точность представления чисел, чем она определяется?

Для целых чисел: Точность абсолютная, так как они представляются без погрешности. Определяется самим фактом использования целого типа.

Для вещественных чисел: Точность (количество верных значащих цифр) определяется длиной мантиссы. Максимально под представление мантиссы отводится 52 разряда, а под представление порядка – 11 разрядов. В этом случае возможные значения чисел находятся в диапазоне от $3.6\,E-4951$ до $1.1\,E+4932$. Если результат операции требует большей точности или мантисса при вычислениях выходит за отведённую разрядную сетку, происходит её округление, что приводит к потере точности.

3. Какие стандартные операции возможны над числами?

Над числами стандартных типов данных (целые и вещественные) возможны все базовые арифметические операции (сложение, вычитание, умножение, деление), а также операции сравнения.

4. Какой тип данных может выбрать программист, если обрабатываемые числа превышают возможный диапазон представления чисел в ПК?

Программист должен выбрать нестандартный тип данных, так как встроенных типов для таких чисел не существует. Наиболее предпочтительным является использование массива (например, массива символов или целых чисел) для хранения цифр числа. Также можно использовать структуру (запись), которая будет хранить отдельно знак мантиссы, саму мантиссу (в виде массива), знак порядка и значение порядка.

5. Как можно осуществить операции над числами, выходящими за рамки машинного представления?

Операции над такими "длинными" числами программист должен реализовать самостоятельно, используя алгоритмы, аналогичные ручным вычислениям "в столбик".

Хранение числа: Представить число в виде массива его цифр. Для повышения эффективности вычислений можно хранить в одном элементе массива не одну цифру, а группу цифр ("тетраду").

Реализация операций:

- Умножение/деление на "короткое" число: Реализуется как последовательное умножение/деление каждой цифры (или тетрады) "длинного" числа с переносом/заёмом старших разрядов в следующий.
- Умножение/деление "длинного" числа на "длинное": Реализуется по алгоритму умножения/деления в столбик, где каждую цифру одного числа умножают на все цифры другого, складывая результаты со сдвигом.